

Obtención de pesos y umbral para las compuertas OR y AND NOT

Implementación mediante la regla de Hebb y el algoritmo del perceptrón

Contenido

1. Introducción
2. Fundamento lógico de las compuertas
3. Modelo de neurona y reglas de aprendizaje
4. Metodología e implementación
5. Resultados obtenidos
6. Análisis de resultados
7. Conclusión

Resumen

En esta práctica se obtuvieron los pesos, el bias y el umbral necesarios para representar las compuertas lógicas OR y x_1 AND NOT x_2 mediante dos modelos de neurona artificial: la regla de Hebb y el perceptrón. La implementación se realizó en Python con PyTorch, usando una codificación bipolar de entradas y salidas. Los cuatro modelos entrenados clasificaron correctamente las cuatro combinaciones posibles de entrada y alcanzaron una exactitud del 100 %.

Resultado principal: OR aprendió pesos positivos para ambas entradas; AND NOT aprendió un peso positivo para x_1 y un peso negativo para x_2 , lo que refleja su función excitadora e inhibidora.

1. Introducción

Las compuertas lógicas pueden modelarse con una neurona artificial porque sus salidas dependen de una combinación ponderada de las entradas. Cada entrada se multiplica por un peso, se agrega un término independiente llamado bias y el resultado se evalúa mediante una función de activación. Cuando la combinación supera el criterio establecido, la neurona produce una salida positiva.

El objetivo del ejercicio es determinar una configuración de pesos y umbral que reproduzca las tablas de verdad de OR y AND NOT. Además, se comparan dos formas de obtener esos parámetros: una actualización directa mediante la regla de Hebb y un entrenamiento iterativo mediante el perceptrón.

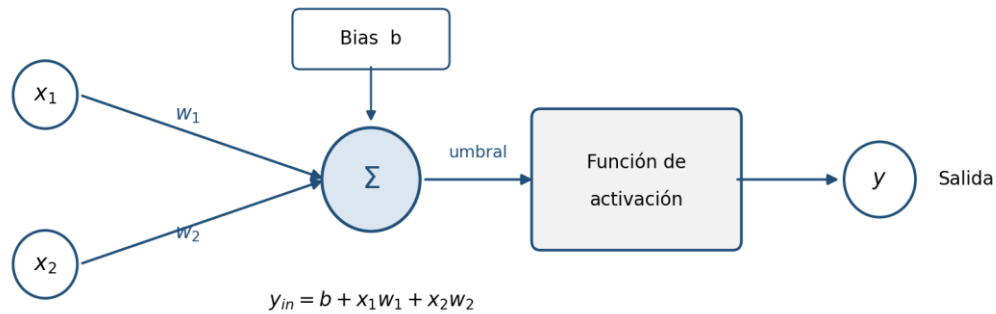


Figura 1. Estructura general de la neurona utilizada

2. Fundamento lógico de las compuertas

2.1 Compuerta OR

La compuerta OR genera una salida igual a 1 cuando al menos una de sus entradas es 1. Únicamente produce 0 cuando las dos entradas son 0.

x_1	x_2	Salida OR
1	1	1
1	0	1
0	1	1
0	0	0

$$y = 1 \text{ si } x_1 + x_2 \geq 1; \text{ en otro caso, } y = 0$$

Una solución binaria equivalente es $w_1 = 1$, $w_2 = 1$ y $\vartheta = 1$.

Los dos pesos son positivos porque cualquiera de las entradas puede activar la salida. Cuando alguna de las entradas toma el valor 1, la suma ponderada alcanza el umbral.

2.2 Compuerta AND NOT

En este ejercicio, AND NOT se interpreta específicamente como x_1 AND NOT x_2 . La salida es 1 solamente cuando x_1 está activada y x_2 está desactivada. En las otras tres combinaciones la salida debe ser 0.

x_1	x_2	Salida x_1 AND NOT x_2
1	1	0
1	0	1
0	1	0
0	0	0

$$y = 1 \text{ si } x_1 - x_2 \geq 1; \text{ en otro caso, } y = 0$$

Una solución binaria equivalente es $w_1 = 1$, $w_2 = -1$ y $\vartheta = 1$.

El peso asociado con x_1 es positivo porque esta entrada favorece la activación. El peso de x_2 es negativo porque x_2 debe inhibir la salida: cuando x_2 vale 1, reduce la entrada neta de la neurona.

2.3 Codificación bipolar utilizada en el programa

Aunque las tablas de verdad se expresan normalmente con 0 y 1, el notebook trabaja con representación bipolar. Por ello, el valor lógico 0 se convierte en -1 y el valor lógico 1 permanece como 1.

Valor lógico	Valor bipolar
0	-1
1	1

El orden de patrones empleado es (1,1), (1,0), (0,1) y (0,0). En forma bipolar, estos patrones se almacenan como (1,1), (1,-1), (-1,1) y (-1,-1). Los targets utilizados fueron $T_{OR} = [1, 1, 1, -1]$ y $T_{AND_NOT} = [-1, 1, -1, -1]$.

3. Modelo de neurona y reglas de aprendizaje

3.1 Entrada neta

La neurona combina las dos entradas con sus pesos y agrega el bias. Esta operación se denomina entrada neta y se expresa de la siguiente manera:

$$y_{in} = b + x_1 w_1 + x_2 w_2$$

x_1 y x_2 son las entradas; w_1 y w_2 son los pesos; b es el bias.

3.2 Regla de Hebb

La regla de Hebb actualiza los pesos utilizando directamente cada patrón de entrada y su salida esperada. En el notebook, los pesos y el bias se inicializan en cero y se recorren una sola vez todos los patrones.

$$w(\text{nuevo}) = w(\text{anterior}) + x \cdot t$$

La multiplicación $x \cdot t$ se aplica a cada componente del vector de entrada.

$$b(\text{nuevo}) = b(\text{anterior}) + t$$

Después del entrenamiento, la función forward calcula y_{in} . Si y_{in} es mayor o igual que 0, la salida es 1; de lo contrario, es -1 . Por ello se considera que Hebb utiliza un umbral implícito de 0.

3.3 Algoritmo del perceptrón

El perceptrón evalúa cada patrón y actualiza sus parámetros únicamente cuando la salida calculada es diferente del target. El proceso se repite por épocas hasta que todos los patrones quedan clasificados correctamente.

$$w(\text{nuevo}) = w(\text{anterior}) + \alpha x$$

α es la tasa de aprendizaje; en el ejercicio se utilizó $\alpha = 1$.

$$b(\text{nuevo}) = b(\text{anterior}) + \alpha t$$

La función de activación del perceptrón usa un umbral simétrico. Con $\theta = 1$, devuelve 1 cuando $y_{in} > 1$, devuelve -1 cuando $y_{in} < -1$ y devuelve 0 cuando el valor queda dentro del intervalo $[-1, 1]$. Después del entrenamiento, ninguno de los patrones analizados queda en la salida 0.

Aclaración: No existe una única combinación válida de pesos. Diferentes valores pueden representar la misma compuerta, siempre que mantengan la separación correcta entre las salidas.

4. Metodología e implementación

La práctica se implementó en un notebook de Jupyter con PyTorch. Se definieron dos clases, HebbNeuron y PerceptronNeuron; posteriormente se prepararon las entradas bipolares, se entrenó una neurona independiente para cada compuerta y se verificaron las cuatro combinaciones de entrada.

- Definición de las clases de neurona y sus métodos de entrenamiento.
- Construcción del conjunto S con las cuatro entradas bipolares.
- Definición de los targets T_OR y T_AND_NOT.
- Entrenamiento de cuatro modelos: OR con Hebb, AND NOT con Hebb, OR con perceptrón y AND NOT con perceptrón.
- Conversión de los resultados bipolares a valores binarios para compararlos con las tablas de verdad.

4.1 Preparación de entradas y salidas

Fragmento 1. Entradas y targets en representación bipolar

```
S = torch.tensor([
    [ 1,  1],    # 1, 1
    [ 1, -1],    # 1, 0
    [-1,  1],    # 0, 1
    [-1, -1]     # 0, 0
])

T_OR = torch.tensor([1, 1, 1, -1])
T_AND_NOT = torch.tensor([-1, 1, -1, -1])
```

4.2 Actualización con la regla de Hebb

Fragmento 2. Regla de actualización de pesos y bias en Hebb

```
for i in range(len(S)):
    x = S[i, :]
    y = T[i]

    self.w = self.w + x * y
    self.b = self.b + y
```

Cada vector de entrada se multiplica por su salida esperada. Si el target es positivo, el patrón se agrega a los pesos; si es negativo, el patrón se resta. El bias se actualiza acumulando los targets.

4.3 Activación y actualización del perceptrón

Fragmento 3. Función de activación con umbral simétrico

```
y_in = torch.matmul(x, self.w)

if y_in > self.threshold:
    y = 1
elif y_in < -self.threshold:
    y = -1
else:
    y = 0
```

Fragmento 4. Actualización del perceptrón cuando existe un error

```
if y != t:
    self.w[1:] = self.w[1:] + self.learning_rate * x_column * t
    self.w[0] = self.w[0] + self.learning_rate * t
```

4.4 Entrenamiento de cada compuerta

Fragmento 5. Entrenamiento independiente de OR y AND NOT

```
perceptron_or = PerceptronNeuron(input_size=2)
perceptron_or.train(
    S=S,
    T=T_OR,
    threshold=1,
    learning_rate=1
)

perceptron_and_not = PerceptronNeuron(input_size=2)
perceptron_and_not.train(
    S=S,
    T=T_AND_NOT,
    threshold=1,
    learning_rate=1
)
```

Se crean objetos diferentes para evitar que los pesos aprendidos por una compuerta se mezclen con los de la otra. Esta separación es necesaria porque OR y AND NOT representan fronteras de decisión distintas.

5. Resultados obtenidos

5.1 Resultados con la regla de Hebb

Compuerta	w_1	w_2	Bias	Umbral	Aciertos	Exactitud
OR	2.0	2.0	2.0	0 implícito	4 de 4	100 %
x_1 AND NOT x_2	2.0	-2.0	-2.0	0 implícito	4 de 4	100 %

Para OR, ambos pesos y el bias son positivos. Esto permite que la presencia de cualquiera de las entradas active la neurona. Para AND NOT, w_1 es positivo y w_2 es negativo, por lo que x_1 impulsa la activación mientras que x_2 la inhibe.

5.2 Resultados con el perceptrón

Compuerta	Épocas	w_1	w_2	Bias	θ	Aciertos	Exactitud
OR	2	2.0	2.0	2.0	1	4 de 4	100 %
x_1 AND NOT x_2	3	2.0	-2.0	-2.0	1	4 de 4	100 %

El perceptrón OR alcanzó una época con actualizaciones y una segunda época de comprobación sin errores. AND NOT necesitó dos épocas con ajustes y una tercera época para confirmar la convergencia. En ambos casos los parámetros finales coincidieron con los obtenidos mediante Hebb, aunque el perceptrón mantuvo un umbral explícito $\theta = 1$.

5.3 Verificación de las tablas de verdad

x_1	x_2	OR esperado	OR obtenido	AND NOT esperado	AND NOT obtenido
1	1	1	1	0	0
1	0	1	1	1	1
0	1	1	1	0	0
0	0	0	0	0	0

La tabla anterior representa los resultados binarios después de convertir las salidas bipolares. Las predicciones de Hebb y del perceptrón fueron iguales para los cuatro patrones, por lo que no se presentó ningún error de clasificación.

6. Análisis de resultados

Los resultados confirman que OR y x_1 AND NOT x_2 son funciones linealmente separables. Esto significa que es posible encontrar una frontera lineal que separe las combinaciones con salida positiva de aquellas con salida negativa. Por esta razón, una sola neurona es suficiente para modelarlas.

- En OR, $w_1 = 2$ y $w_2 = 2$ indican que ambas entradas tienen el mismo efecto positivo sobre la salida.
- En AND NOT, $w_1 = 2$ y $w_2 = -2$ expresan que x_1 es excitadora y x_2 es inhibidora.
- El bias de OR es positivo, lo que desplaza la activación hacia la clase positiva en la representación bipolar.

- El bias de AND NOT es negativo, lo que hace más restrictiva la activación y permite que sólo el patrón (1,0) produzca 1.
- Hebb calculó los parámetros en un único recorrido; el perceptrón verificó y corrigió los errores por épocas hasta converger.

Aunque los pesos finales fueron 2 y -2, esto no contradice la solución lógica manual con pesos 1 y -1. Las dos configuraciones son equivalentes porque conservan las relaciones de signo y la separación entre las clases. Los valores exactos dependen de la codificación, el bias, el umbral, la tasa de aprendizaje y el orden en que se presentan los patrones.

Limitación del ejercicio: La exactitud del 100 % se calculó sobre los mismos cuatro patrones utilizados para definir y entrenar las compuertas. En este caso es adecuado porque la tabla de verdad contiene todo el universo posible de entradas binarias, pero no debe interpretarse como una evaluación de generalización sobre datos desconocidos.

7. Conclusión

La práctica permitió representar correctamente las compuertas OR y x_1 AND NOT x_2 mediante neuronas artificiales. Primero se analizaron sus tablas de verdad y se identificó el efecto esperado de cada entrada. Después se aplicó una codificación bipolar y se entrenaron modelos con la regla de Hebb y con el perceptrón.

Los dos métodos obtuvieron una exactitud del 100 %. Para OR se encontraron $w_1 = 2$, $w_2 = 2$ y $b = 2$; para AND NOT se obtuvieron $w_1 = 2$, $w_2 = -2$ y $b = -2$. En Hebb se utilizó un umbral implícito de 0, mientras que en el perceptrón se configuró $\theta = 1$. El signo de los pesos refleja directamente la lógica de cada compuerta: OR requiere contribuciones positivas de ambas entradas, mientras que AND NOT necesita que x_2 inhiba la activación.

En conclusión, el ejercicio demuestra que una neurona simple puede aprender funciones lógicas linealmente separables y que las reglas de aprendizaje transforman una tabla de verdad en parámetros numéricos capaces de reproducirla. También muestra que distintas combinaciones de pesos pueden ser válidas si generan las mismas salidas para todas las entradas posibles.